known as SSL). You will learn how to enable HTTPS for an API in chapter 2.

Finally, from application security you will learn

- Secure coding techniques
- Common software security vulnerabilities
- How to store and manage system and user credentials used to access your APIs

1.3.1 A typical API deployment

An API is implemented by application code running on a server; either an *application server* such as Java Enterprise Edition (Java EE), or a standalone server. It is very rare to directly expose such a server to the internet, or even to an internal intranet. Instead, requests to the API will typically pass through one or more additional network services before they reach your API servers, as shown in figure 1.3. Each request will pass through one or more *firewalls*, which inspect network traffic at a relatively low level and ensure that any unexpected traffic is blocked. For example, if your APIs are serving requests on port 80 (for HTTP) and 443 (for HTTPS), then the firewall would be configured to block any requests for any other ports. A *load balancer* will then route traffic to appropriate services and ensure that one server is not overloaded with lots of requests while others sit idle. Finally, a *reverse proxy* (or *gateway*) is typically placed in front of the application servers to perform computationally expensive operations like handling TLS/SSL encryption and validating credentials on requests.